

ADA - Análisis y Diseño de Algoritmos, 2026-1**Tarea 3: Semanas 8 y 9**

Para entregar el lunes 6 de abril de 2025

Problemas conceptuales a las 23:59 por BrightSpace

Problemas prácticos a las 23:59 en la arena de programación

Tanto los ejercicios como los problemas deben ser resueltos, pero únicamente las soluciones de los problemas deben ser entregadas. La intención de los ejercicios es entrenarlo para que domine el material del curso; a pesar de que no debe entregar soluciones a los ejercicios, usted es responsable del material cubierto en ellos.

Instrucciones para la entrega

Para esta tarea y todas las tareas futuras, la entrega de soluciones es *individual*. Por favor escriba claramente su nombre, código de estudiante y sección en cada hoja impresa entregada o en cada archivo de código (a modo de comentario). Adicionalmente, agregue la información de fecha y nombres de compañeros con los que colaboró; igualmente cite cualquier fuente de información que utilizó.

¿Cómo describir un algoritmo?

En algunos ejercicios y problemas se pide “dar un algoritmo” para resolver un problema. Una solución debe tomar la forma de un pequeño ensayo (es decir, un par de párrafos). En particular, una solución debe resumir en un párrafo el problema y cuáles son los resultados de la solución. Además, se deben incluir párrafos con la siguiente información:

- una descripción del algoritmo en castellano y, si es útil, pseudo-código;
- por lo menos un diagrama o ejemplo que muestre cómo funciona el algoritmo;
- una demostración de la corrección del algoritmo; y
- un análisis de la complejidad temporal del algoritmo.

Recuerde que su objetivo es comunicar claramente un algoritmo. Las soluciones algorítmicas correctas y descritas *claramente* recibirán alta calificación; soluciones complejas, obtusas o mal presentadas recibirán baja calificación.

Problemas conceptuales

1. Ejercicio 4.7: *El Goog* (Kleinberg & Tardos, página 191).
2. Ejercicio 22: *Balanced Parentheses* (Erickson, página 184).

Problemas prácticos

Hay cinco problemas prácticos cuyos enunciados aparecen a partir de la siguiente página.

A - Calculus Simplified

Source file name: calculus.py

Time limit: x seconds

We define an expression as below:

$$\langle expression \rangle := \langle number \rangle \mid \langle expression \rangle + \langle expression \rangle \mid \langle expression \rangle - \langle expression \rangle \mid (\langle expression \rangle)$$

where $\langle number \rangle$ is defined to be an integer.

In this problem you are given an expression with all its numbers replaced with character 'x'. Then you are given a set of numbers that were actually in the expression. We know that numbers were placed in the expression in such a way that the expression evaluates to maximum possible value among all other placements. Write a program that calculates this maximum value.

Input

In the first line there is an integer T ($T \leq 100$), the number of tests. Each test begins with the expression itself. Next line is an integer N , the number of numbers in the expression. In the final line of each test there are N integers a_i ($\|a_i\| \leq 3000$). Each of these numbers should be used in the expression exactly once. It is guaranteed that the expression can be parsed by the definition in the problem statement and its length will not exceed 10^5 . There are no whitespaces in the expression and all numbers are replaced with a single 'x'. The number of 'x's in the expression is N .

The input must be read from standard input.

Output

For each test output the maximum possible value of the expression in a single line.

The output must be written to standard output.

Sample Input	Sample Output
3	2
x	2
1	1
2	
x-x	
2	
-1 1	
(x)+(x)-(x)	
3	
1 1 1	

B - Making Change

Source file name: `change.py`

Time limit: x seconds

Given an amount of money and unlimited (almost) numbers of coins (we will ignore notes for this problem) we know that an amount of money may be made up in a variety of ways. A more interesting problem arises when goods are bought and need to be paid for, with the possibility that change may need to be given. Given the finite resources of most wallets nowadays, we are constrained in the number of ways in which we can make up an amount to pay for our purchases —assuming that we can make up the amount in the first place, but that is another story.

The problem we will be concerned with will be to minimise the number of coins that change hands at such a transaction, given that the shopkeeper has an adequate supply of all coins (the set of New Zealand coins comprises 5c, 10c, 20c, 50c, \$1 and \$2). Thus, if we need to pay 55c and we do not hold a 50c coin, we could pay this as $2 * 20c + 10c + 5c$ to make a total of 4 coins. If we tender \$1 we will receive 45c in change which also involves 4 coins, but if we tender \$1.05 (\$1 + 5c), we get 50c change and the total number of coins that changes hands is only 3.

Write a program that will read in the resources available to you and the amount of the purchase and will determine the minimum number of coins that change hands.

Input

Input will consist of a series of lines, each line defining a different situation. Each line will consist of 6 integers representing the numbers of coins available to you in the order given above, followed by a real number representing the value of the transaction, which will always be less than \$5.00. The file will be terminated by six zeroes (0 0 0 0 0 0). The total value of the coins will always be sufficient to make up the amount and the amount will always be achievable, that is it will always be a multiple of 5c.

The input must be read from standard input.

Output

Output will consist of a series of lines, one for each situation defined in the input. Each line will consist of the minimum number of coins that change hands right justified in a field 3 characters wide.

The output must be written to standard output.

Sample Input	Sample Output
2 4 2 2 1 0 0.95	2
2 4 2 0 1 0 0.55	3
0 0 0 0 0 0	

C - Keep the Customer Satisfied

Source file name: customer.py

Time limit: x seconds

Simon and Garfunkel Corporation (SG Corp.) is a large steel-making company with thousand of customers. Keeping the customer satisfied is one of the major objective of Paul and Art, the managers.

Customers issue orders that are characterized by two integer values q , the amount of steel required (in tons), and d , the due date (a calendar date converted in seconds). The due date has to be met if SG Corp. accepts the order. Stated another way, when an order is accepted, the corresponding amount of steel has to be produced before its due date. Of course, the factory can process no more than one order at a time.

Although the manufacturing process is rather complex, it can be seen as a single production line with a constant throughput. In the following, we assume that producing q tons of steel takes exactly q seconds (i.e., throughput is 1). The factory runs on a monthly production plan. Before the beginning of the month, all customers' orders are collected and Paul and Art determine which of them are going to be accepted and which ones are to be rejected in the next production period. A production schedule is then designed. To keep customers satisfied, Paul and Art want to minimize the total number of orders that are rejected. In the following, we assume that the beginning of the next production plan (i.e., the first day of the next month) corresponds to date 0.

Consider the following data set made of 6 orders $J_1, \dots, J_6$. For a given order, J_j , q_j denotes the amount of steel required and d_j is the associated due date.

Order	q_j	d_j
J_1	6	8
J_2	4	9
J_3	7	15
J_4	8	20
J_5	3	21
J_6	5	22

You can check by hand that all orders cannot be accepted and it's very unlikely you could find a solution with less than two rejected orders. Here is an optimal solution: Reject J_1 and J_4 , accept all other orders, and process them as follows.

Accepted Order	Starting Time	Completion Time
J_2	0	4
J_3	4	11
J_5	11	14
J_6	14	19

Note that the production line is never idle.

Hogdson and Moore have been appointed as Chief Scientific Officers and you are requested to help them to compute an optimal solution and to build a schedule of all accepted orders (starting time and completion time).

Input

The input begins with a single positive integer on a line by itself indicating the number of the cases following, each of them as described below. This line is followed by a blank line, and there is also a blank line between two consecutive inputs. In each test case, the first line contains the number n of orders (n can be as large as 800 000). It is followed by n lines. Each of which describes an order made of two integer values: the amount of steel (in tons) required for the order (lower than 1 000) and its due date (in seconds; lower than 2×10^6).

The input must be read from standard input.

Output

For each test case, you are required to compute an optimal solution and your program has to write the number of orders that are accepted. The outputs of two consecutive cases will be separated by a blank line.

The output must be written to standard output.

Sample Input	Sample Output
1 6 7 15 8 20 6 8 4 9 3 21 5 22	4

D - Lift Hopping in the Real World

Source file name: `lift.py`

Time limit: x seconds

A skyscraper has no more than 100 floors, numbered from 0 to 99. It has n ($1 \leq n \leq 50$) elevators which travel up and down at (possibly) different speeds. For each i in $\{1, 2, \dots, n\}$, elevator number i takes T_i ($1 \leq T_i \leq 100$) seconds to travel between any two adjacent floors (going up or down). Elevators do not necessarily stop on every floor. What's worse, not every floor is necessarily accessible by an elevator.

You are on floor 0 and would like to get to floor k as quickly as possible. When you first arrive to floor 0, each of the n elevators could be parked on any of the floors accessible by that elevator. You can take any of the elevators that stop on floor 0 and you can switch elevators on any floor, f , if both of them stop on floor f . Assume that it takes 5 seconds to exit one elevator and push another elevator's button. You are forbidden from using the staircase. No one else is in the building, so you don't have to stop if you don't want to, and no one else will request an elevator. Each elevator has its own button, and the system has built-in security to prevent abuse by evil teenagers — once you press the button calling for some elevator E , all the buttons calling for other elevators will be disabled until elevator E arrives at your floor. In other words, you can only call for one elevator at a time.

Calculate the number of seconds required to get from floor 0 to floor k in the worst case if you do not know the initial positions of the elevators. Passing floor k while inside an elevator that does not stop there does not count as "getting to floor k ". See the examples below.

Input

The input will consist of a number of test cases. Each test case will begin with two numbers, n and k , on a line. The next line will contain the numbers $T_1, T_2, \dots, T_n$. The next n lines will contain sorted lists of integers — the first line will list the floors visited by elevator number 1, the next one — those visited by elevator number 2, etc.

The input must be read from standard input.

Output

For each test case, output one number on a line — the number of seconds required to get to floor k from floor 0 in the worst case. If floor k is inaccessible from floor 0, print 'IMPOSSIBLE' instead.

The output must be written to standard output.

Discussion. In the first example, the worst case is when elevator 1 is on floor 99 and will take 990 seconds to reach floor 0. You will then take it to floor 13 in 130 seconds, spend 5 second exiting and calling elevator 2, which is on floor 30. It will take 170 seconds to reach you and 170 seconds to take you to floor 30. The total is 1295 seconds.

In the second example, the only sensible way to get to floor 30 is to ride the first elevator all the way. Switching to elevator 2 and then back will only make things worse because elevator 1 must take at least 300 seconds to reach floor 30. In the worst case, elevator 1 starts on floor 30.

In example 3, the security system will not allow you to push all 3 buttons and see which elevator comes first. Instead, you should push elevator 2's button. Unfortunately, it is on floor 30 and takes 1500 seconds to get to you. You board it and it takes you to floor 20 in 1000 seconds. You spend 5 seconds pushing the button for elevator 3, which is on floor 50. It takes 3000 seconds before it reaches floor 20 and 3000 more seconds to take you to floor 50. The total is 8505 seconds. If you take elevator 1 first, your worst case time will be 8710 seconds.

In the last example, the one elevator does not stop at floor 1.

Sample Input	Sample Output
2 30	1295
10 5	600
0 1 3 5 7 9 11 13 15 20 99	8505
4 13 15 19 20 25 30	IMPOSSIBLE
2 30	
10 1	
0 5 10 12 14 20 25 30	
2 4 6 8 10 12 14 22 25 28 29	
3 50	
10 50 100	
0 10 30 40	
0 20 30	
0 20 50	
1 1	
2	
0 2 4 6 8 10	

E - Racing

Source file name: `racing.py`

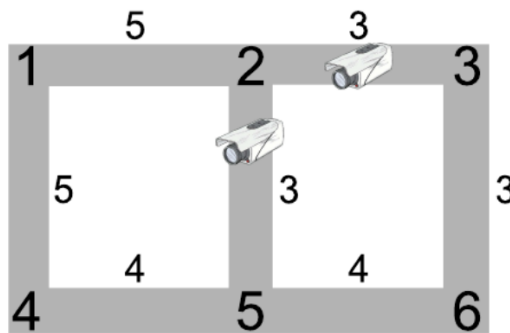
Time limit: 2 seconds

Singapore will host a Formula One race in 2008. The race will be held on a 5.067km long street circuit, consisting of 14 left hand turns and 10 right hand turns. In the run up to the F1 race, the number of illegal night street racing activities have been on the rise. Such races consists of several rounds around a designated street circuit.

The authorities would like to deploy a new vehicle monitoring system in order to catch these illegal Saint Andrew's Road, part of the Formula One circuit racers. The system consists of a (Kenny Pek, Piccom) number of cameras mounted along various roads. For the system to be effective, there should be at least one camera along each of the possible circuits.

The Singapore road system can be represented as a series of junctions and connecting bidirectional roads. A possible racing circuit consists of a start junction followed by a path consisting of three or more roads that eventually leads back to the start junction. Each road in a racing circuit can be traversed only in one direction, and only once.

Your task is to write a program that computes the optimal placement of the vehicle-monitoring cameras. You will be provided with a description of a connected road network to be monitored in terms of the roads and junctions. The junctions are identified by the bigger numbers in the figure below. A camera can be deployed on the roads (and not the junctions).



The cost of deploying a camera depends on the road on which it is placed. The smaller numbers by the roads in the figure indicate the cost of deploying a camera on that road. Your job is to select a set of roads that minimizes the total cost of deployment while ensuring that there is at least one camera along every possible racing circuit (i.e., loop in the road network).

Input

The input consists of a line containing the number c of datasets, followed by c datasets, followed by a line containing the number '0'.

The first line of each dataset contains two positive integers, n and m , separated by a blank, which represent the number of junctions and number of roads, respectively. You may assume that $0 < n < 10\,000$ and $0 < m < 100\,000$. For simplicity, each one of the n junctions are labeled from 1 to n . The following m lines of each dataset each describes one road. Each line consists of three positive integers which are the labels of two different junctions and the cost of deploying a camera on this road. The cost of deploying a camera is between 1 and 1 000.

The input must be read from standard input.

Output

The output consists of one line for each dataset. The c -th line contains one single number, representing the minimal cost of setting up the vehicle monitoring system such that there is at least one camera along every possible circuit.

The output must be written to standard output.

Sample Input	Sample Output
1 6 7 1 2 5 2 3 3 1 4 5 4 5 4 5 6 4 6 3 3 5 2 3 0	6